

En la práctica 8, se daban las estructuras siguientes:

```
struct Tfecha {  
    int dia, mes, anio;  
};  
  
struct Tcliente {  
    char apellidos[40+1];  
    struct Tfecha antigüedad;  
    float ultimaCompra;  
};  
  
struct TlistaClientes{  
    int numClientes;  
    struct Tcliente arrayClientes[N];  
};
```

para definir una lista de clientes. Utilizando las funciones definidas en la práctica anterior, se deben implementar las funciones descritas en la sección de prototipos, probando su funcionamiento en un programa.

SOLUCIÓN:**// includes y defines**

```
#include <stdio.h>
#include <stdlib.h>
#define N 200
```

// estructuras

```
struct Tfecha {
    int dia, mes, anio;
};
```

```
struct Tcliente {
    char apellidos[40+1];
    struct Tfecha antiguedad;
    float ultimaCompra;
};
```

```
struct TlistaClientes{
    int numClientes;
    struct Tcliente arrayClientes[N];
};
```

// prototipos

```
void scanFecha(struct Tfecha *);
void printFecha (struct Tfecha);
void scanCliente (struct Tcliente *);
void printCliente (struct Tcliente);
void printListaClientes (struct TlistaClientes);
void inicializarLista (struct TlistaClientes *);
void anadirCliente (struct TlistaClientes *);
int longitudLista (struct TlistaClientes);
```

```
void copiarFechas (struct Tfecha *, struct Tfecha *);
// copia la segunda fecha en la primera fecha
```

```
void copiarClientes (struct Tcliente *, struct Tcliente *);
// copia el segundo cliente en el primer cliente
```

```
void insertarCliente (struct TlistaClientes *, int);
// inserta un cliente, usando scanCliente, en una posicion que es un integer
```

```
void eliminarCliente (struct TlistaClientes *, int);
```

```
// elimina un cliente, de una posicion que es un integer
```

```
void vaciarListaClientes (struct TlistaClientes *);  
// elimina todos los clientes de la lista
```

```
// main  
int main()  
{  
    struct TlistaClientes lista;  
    struct Tfecha fecha1;  
    struct Tfecha fecha2;  
    longitudLista(lista);  
    copiarFechas(&fecha1,&fecha2);  
    printListaClientes(lista);  
    return 0;  
}
```

```
// implementacion de las funciones
```

```
void scanFecha(struct Tfecha *fecha){  
    printf("dia? ");  
    scanf("%d",&fecha->dia);  
    printf("mes? ");  
    scanf("%d",&fecha->mes);  
    printf("anio? ");  
    scanf("%d",&fecha->anio);  
}  
  
void printFecha (struct Tfecha fecha){  
    printf("\nantigüedad: %d-%d-%d",fecha.dia,fecha.mes,fecha.anio);  
}  
  
void scanCliente (struct Tcliente *cliente){  
    printf("apellidos? ");  
    scanf("%s",&cliente->apellidos);  
    scanFecha(&cliente->antigüedad);  
    printf("ultima compra: ");  
    scanf("%f",&cliente->ultimaCompra);  
}  
  
void printCliente (struct Tcliente cliente){  
    struct Tfecha antigüedad;  
    printf("\napellidos: %s",cliente.apellidos);  
    fflush(stdin);  
    printFecha(cliente.antigüedad);  
    printf("\nultima compra: %.2f",cliente.ultimaCompra);
```

```

}

void printListaClientes (struct TlistaClientes clientes){
    for(int i=0;i<clientes.numClientes;i++)
        printCliente(clientes.arrayClientes[i]);
}

void inicializarLista (struct TlistaClientes *clientes){
    clientes->numClientes=1;
}

void anadirCliente (struct TlistaClientes *clientes){
    int i;
    struct Tcliente arrayClientes[N];
    char letra;
    scanCliente(&clientes->arrayClientes[0]);
    inicializarLista(&clientes->numClientes);
    for(i=1;i<=clientes->numClientes;i++){
        printf("anadir cliente?");
        fflush(stdin);
        scanf("%c",&letra);
        if(letra == 's'){
            scanCliente(&clientes->arrayClientes[i]);
            (clientes->numClientes)++;
        }else if(letra == 'n'){
            printListaClientes(*clientes);
            return 0;
        }
    }
}

int longitudLista (struct TlistaClientes clientes){
    if(clientes.numClientes < N)
        anadirCliente(&clientes);
}

//practica 9

void copiarFechas (struct Tfecha *primero, struct Tfecha *segundo){
    primero->anio = segundo->anio;
    primero->mes = segundo->mes;
    primero->dia = primero->dia;
}

void copiarClientes (struct Tcliente *primero, struct Tcliente *segundo){
    strcpy(primero->apellidos,segundo->apellidos);
    copiarFechas(&primero->antiguedad,&segundo->antiguedad);
    primero->ultimaCompra = segundo->ultimaCompra;
}

```

```
}
```

```
void insertarCliente (struct TlistaClientes *lista, int n){  
    if(n<= longitudLista(*lista)){  
        if(longitudLista(*lista) < N){  
            scanCliente(&lista->arrayClientes[n]);  
            lista->numClientes++;  
        }  
    }else if (n>=N)  
        printf("ERROR");  
}
```

```
void eliminarCliente(struct TlistaClientes *lista, int n){  
    if(n<= longitudLista(*lista)){  
        if(longitudLista(*lista) < N){  
            for(int i=n;i< lista->numClientes;i++){  
                lista->arrayClientes [i]= lista->arrayClientes[i+1];  
            }  
            lista->numClientes--;  
        }  
    }  
}
```

```
void vaciarListaClientes(struct TlistaClientes *lista){  
    do{  
        for(int i=0; i < lista->numClientes; i++){  
            lista->arrayClientes[i] = lista->arrayClientes[i+1];  
        }  
        lista->numClientes--;  
    }while(lista->numClientes == 0);  
}
```